

SSE

Sistema de Soporte a la Ensenanza - Reactor Nuclear RA-0

Estructura base del monorepo — arbol de carpetas y justificacion

Proyecto Integrador - Ingenieria en Computacion - UNC / CUTN

Este documento describe la estructura de archivos y carpetas propuesta para el monorepo del SSE (backend FastAPI + frontend React/TypeScript), la justificacion de cada decision de organizacion, y que partes del stack estan **confirmadas** por el usuario frente a las que quedan **asumidas** a falta de definicion.

El repositorio real ya fue generado en D:\Facultad\Tesis (166 archivos, git inicializado) a partir de esta propuesta.

1. Stack tecnologico confirmado

Arquitectura monorepo. Backend Python + FastAPI, organizado como API modular monolitica por dominios funcionales. Comunicacion REST + WebSockets. Base de datos PostgreSQL. Frontend React + TypeScript, cliente unicamente web. Autenticacion JWT propio (access token + refresh token), sin sistema de roles: todos los usuarios tienen el mismo nivel de privilegio. Despliegue local/desarrollo con Docker Compose. Tests unitarios solamente (sin integracion/E2E confirmados).

Regla de negocio central: la senal **SERMO** habilita/deshabilita el modulo de tiempo real (RF04); cuando SERMO=false quedan disponibles los historicos, incluyendo la operacion recién finalizada.

Modelo de datos operativos: tabla ancha — una fila por muestra temporal (~1 muestra/segundo), una columna por variable (~30 variables). Misma estructura para datos en tiempo real e historicos.

2. Arbol completo de carpetas y archivos

Arbol real del monorepo generado (166 archivos). Las carpetas terminadas en / son directorios.

```
|-- .vscode/
|   |-- extensions.json
|   |-- settings.json
|-- apps/
|   |-- backend/
|   |   |-- app/
|   |   |   |-- api/
|   |   |   |   |-- v1/
|   |   |   |   |   |-- __init__.py
|   |   |   |   |   |-- router.py
|   |   |   |   |-- __init__.py
|   |   |   |-- config/
|   |   |   |   |-- __init__.py
|   |   |   |   |-- logging.py
|   |   |   |   |-- settings.py
|   |   |   |-- core/
|   |   |   |   |-- __init__.py
|   |   |   |   |-- exceptions.py
|   |   |   |   |-- middleware.py
|   |   |   |   |-- security.py
|   |   |   |-- db/
|   |   |   |   |-- __init__.py
|   |   |   |   |-- base.py
|   |   |   |   |-- session.py
|   |   |   |-- domains/
|   |   |   |   |-- audit/
|   |   |   |   |   |-- __init__.py
|   |   |   |   |   |-- models.py
|   |   |   |   |   |-- repository.py
|   |   |   |   |   |-- service.py
|   |   |   |   |-- auth/
|   |   |   |   |   |-- adapters/
|   |   |   |   |   |   |-- sso/
|   |   |   |   |   |   |   |-- __init__.py
|   |   |   |   |   |   |   |-- README.md
|   |   |   |   |   |   |-- __init__.py
|   |   |   |   |   |   |-- base.py
|   |   |   |   |   |   |-- external_api_provider.py
|   |   |   |   |   |   |-- mock_provider.py
|   |   |   |   |   |-- __init__.py
|   |   |   |   |   |-- dependencies.py
|   |   |   |   |   |-- models.py
|   |   |   |   |   |-- repository.py
|   |   |   |   |   |-- router.py
|   |   |   |   |   |-- schemas.py
|   |   |   |   |   |-- service.py
|   |   |   |   |-- exports/
|   |   |   |   |   |-- __init__.py
|   |   |   |   |   |-- router.py
|   |   |   |   |   |-- schemas.py
```

```

|-- service.py
|-- help/
|   |-- __init__.py
|   |-- router.py
|-- historicals/
|   |-- __init__.py
|   |-- models.py
|   |-- repository.py
|   |-- router.py
|   |-- schemas.py
|   |-- service.py
|-- reactor_data/
|   |-- __init__.py
|   |-- models.py
|   |-- schemas.py
|   |-- variable_catalog.py
|-- reactor_state/
|   |-- __init__.py
|   |-- router.py
|   |-- schemas.py
|   |-- service.py
|-- realtime/
|   |-- __init__.py
|   |-- router.py
|   |-- schemas.py
|   |-- service.py
|-- reports/
|   |-- __init__.py
|   |-- models.py
|   |-- router.py
|   |-- schemas.py
|   |-- service.py
|-- __init__.py
|-- websocket/
|   |-- __init__.py
|   |-- dependencies.py
|   |-- events.py
|   |-- manager.py
|-- __init__.py
|-- main.py
|-- migrations/
|-- versions/
|   |-- .gitkeep
|-- env.py
|-- script.py.mako
|-- tests/
|   |-- domains/
|   |   |-- auth/
|   |   |   |-- __init__.py
|   |   |   |-- test_service.py
|   |   |-- historicals/
|   |   |   |-- __init__.py
|   |   |   |-- test_service.py
|   |   |-- reactor_state/
|   |   |   |-- __init__.py
|   |   |   |-- test_service.py
|   |   |-- __init__.py
|   |-- __init__.py
|   |-- conftest.py
|-- .env.example
|-- alembic.ini
|-- pyproject.toml
|-- pytest.ini
|-- README.md
|-- frontend/
|   |-- src/
|   |   |-- app/
|   |   |   |-- providers/
|   |   |   |   |-- AuthProvider.tsx
|   |   |   |   |-- QueryProvider.tsx
|   |   |   |-- AppShell.tsx
|   |   |   |-- routes.tsx
|   |-- components/
|   |-- charts/

```

```

|-- adapters/
|   |-- placeholder/
|   |   |-- PlaceholderChartAdapter.tsx
|   |-- recharts/
|   |   |-- .gitkeep
|   |   |-- README.md
|   |-- activeAdapter.ts
|   |-- ChartRenderer.test.tsx
|   |-- ChartRenderer.tsx
|   |-- README.md
|   |-- types.ts
|-- icons/
|   |-- index.tsx
|-- ui/
|   |-- Button.tsx
|   |-- Card.tsx
|   |-- Field.tsx
|-- features/
|   |-- about/
|   |   |-- pages/
|   |   |   |-- AboutPage.tsx
|   |-- auth/
|   |   |-- hooks/
|   |   |   |-- useAuth.ts
|   |   |-- pages/
|   |   |   |-- LoginPage.test.tsx
|   |   |   |-- LoginPage.tsx
|   |   |-- services/
|   |   |   |-- authApi.ts
|   |   |-- types.ts
|   |-- help/
|   |   |-- pages/
|   |   |   |-- HelpPage.tsx
|   |-- historicals/
|   |   |-- pages/
|   |   |   |-- HistoricalsPage.tsx
|   |   |   |-- OperationDetailPage.tsx
|   |   |-- services/
|   |   |   |-- historicalsApi.ts
|   |-- home/
|   |   |-- pages/
|   |   |   |-- HomePage.tsx
|   |-- realtime/
|   |   |-- components/
|   |   |   |-- ControlBarsPanel.tsx
|   |   |-- hooks/
|   |   |   |-- useReactorState.ts
|   |   |   |-- useRealtimeSocket.ts
|   |   |-- pages/
|   |   |   |-- RealtimePage.tsx
|   |   |-- services/
|   |   |   |-- realtimeApi.ts
|   |-- reports/
|   |   |-- pages/
|   |   |   |-- ReportsPage.tsx
|   |   |-- services/
|   |   |   |-- reportsApi.ts
|-- i18n/
|   |-- locales/
|   |   |-- en/
|   |   |   |-- common.json
|   |   |-- es/
|   |   |   |-- common.json
|   |-- index.ts
|-- mocks/
|   |-- operations.mock.ts
|   |-- variables.mock.ts
|-- services/
|   |-- api/
|   |   |-- endpoints.ts
|   |   |-- httpClient.ts
|   |-- websocket/
|   |   |-- topics.ts
|   |   |-- wsClient.ts

```

```

|         |         |-- store/
|         |         |-- reactorStateStore.ts
|         |         |-- styles/
|         |         |   |-- globals.css
|         |         |   |-- tokens.css
|         |         |-- types/
|         |         |   |-- operation.ts
|         |         |   |-- reactor.ts
|         |         |-- App.tsx
|         |         |-- main.tsx
|         |         |-- vite-env.d.ts
|         |-- .env.example
|         |-- index.html
|         |-- package.json
|         |-- postcss.config.js
|         |-- README.md
|         |-- tailwind.config.ts
|         |-- tsconfig.json
|         |-- vite.config.ts
|-- docs/
|   |-- api/
|   |   |-- README.md
|   |-- architecture/
|   |   |-- data-model.md
|   |   |-- overview.md
|   |   |-- websocket-protocol.md
|   |-- decisions/
|   |   |-- 0001-record-architecture-decisions.md
|   |   |-- 0002-chart-library-deferred.md
|   |-- design/
|   |   |-- frontend-prototype/
|   |   |   |-- assets/
|   |   |   |   |-- ra0-control-room.png
|   |   |   |-- src/
|   |   |   |   |-- About.jsx
|   |   |   |   |-- App.jsx
|   |   |   |   |-- ChartPlaceholder.jsx
|   |   |   |   |-- ControlBarsPanel.jsx
|   |   |   |   |-- Help.jsx
|   |   |   |   |-- Histories.jsx
|   |   |   |   |-- Home.jsx
|   |   |   |   |-- icons.jsx
|   |   |   |   |-- labels.jsx
|   |   |   |   |-- Login.jsx
|   |   |   |   |-- mockData.jsx
|   |   |   |   |-- Realtime.jsx
|   |   |   |   |-- Shell.jsx
|   |   |   |   |-- Suggestions.jsx
|   |   |-- uploads/
|   |   |   |-- 1.png
|   |   |   |-- 2.png
|   |   |   |-- 3.png
|   |   |   |-- 4.png
|   |   |   |-- 5.png
|   |   |   |-- 6.png
|   |   |   |-- 7.png
|   |   |   |-- draw-07600d5e-0a1b-4d20-a5a8-3ebba7cb497c.png
|   |   |   |-- draw-12f74df3-99ab-4e9a-a9d2-83a3ffb5d90a.png
|   |   |   |-- draw-26b8a36c-4df5-4321-aca5-9f64a0e2327b.png
|   |   |   |-- draw-876cbe5b-db8c-441f-bf5c-72f83100b8d4.png
|   |   |   |-- draw-b614dbab-ae66-4a67-b6e9-ab66dfa680d0.png
|   |   |   |-- draw-d069f558-e132-48ec-9705-a4414bea4018.png
|   |   |   |-- draw-f0f2b233-81e5-4a9c-b8b6-62073d5013fa.png
|   |   |   |-- im reactor.png
|   |   |-- .thumbnail
|   |   |-- README.md
|   |   |-- SSE.html
|   |-- manual-usuario/
|   |   |-- README.md
|-- infra/
|   |-- docker/
|   |   |-- postgres/
|   |   |   |-- README.md
|   |-- backend.Dockerfile

```

```
| | `-- frontend.Dockerfile
| |-- nginx/
| |-- README.md
|-- scripts/
| |-- db/
| | |-- seed.py
| | |-- wait_for_db.py
| |-- dev/
| | |-- down.ps1
| | |-- logs.ps1
| | |-- up.ps1
|-- .editorconfig
|-- .env.example
|-- .gitignore
|-- docker-compose.yml
|-- README.md
`-- sse-ra0.code-workspace
```

Nota: se omiten del arbol los archivos generados por herramientas (cache de Python, node_modules) y la carpeta .git.

3. Proposito de cada carpeta principal

Carpeta	Proposito
apps/backend	API FastAPI. Todo el codigo Python del sistema, organizado por dominios funcionales (no por capas tecnicas).
apps/frontend	SPA React + TypeScript. Organizada por features que reflejan los dominios del backend.
infra/docker	Dockerfiles de backend y frontend, centralizados fuera de apps/ para mantener las apps libres de configuracion de despliegue.
infra/nginx	Reservada para un reverse proxy de produccion (TLS, servir el build estatico). No forma parte del alcance confirmado hoy.
docs/architecture	Vision general de la arquitectura, modelo de datos y contrato de WebSocket. Documenta explicitamente que esta confirmado vs. asumido.
docs/decisions	ADRs (Architecture Decision Records) numerados: registran el porque de cada decision tecnica, no solo el que.
docs/design/frontend-prototype	Copia del prototipo de Claude Design (SSE V1.zip) entregado por el usuario, preservado como referencia de diseno versionada junto al codigo.
docs/manual-usuario	Placeholder del entregable exigido por RF07 (Manual de Usuario).
scripts/dev	Wrappers cortos de Docker Compose (up/down/logs) en PowerShell.
scripts/db	Utilidades de base de datos: espera de disponibilidad, seed de datos de desarrollo.

apps/backend/app/domains/ (uno por dominio)

Dominio	Responsabilidad
auth	Login, JWT (access+refresh), control de intentos fallidos, adapters de credenciales (externo hoy, SSO reservado a futuro).
reactor_data	Catalogo de variables del reactor y schemas base compartidos por realtime e historicals.
reactor_state	Fuente unica de verdad de SERMO/estado de operacion. REST + WebSocket.
realtime	Stream de muestras via WebSocket, habilitado solo si reactor_state indica OPERACION.
historicals	Consulta de operaciones pasadas por fecha o ID, descarga de datos.
exports	Registro de auditoria de exportaciones (la generacion del PNG se asume client-side).
reports	Sugerencias y reportes de error (RF08).
help	Metadata del manual de usuario (RF07).
audit	Servicio transversal de logs: accesos, errores, exportaciones, consultas, conexiones WebSocket (RNF03).

4. Justificacion de las decisiones de organizacion

- **Backend por dominios, no por capas:** cada carpeta en domains/ agrupa router, service, schemas, models y repository de un mismo concepto de negocio. No existen carpetas transversales controllers/, services/, models/ a nivel raiz. Esto hace que el "modulo monolitico" sea real: cada dominio podria extraerse a su propio servicio en el futuro moviendo solo esa carpeta.
- **No se agrego packages/:** se evaluo para contratos TypeScript compartidos generados desde el OpenAPI del backend, pero con solo dos aplicaciones y sin cliente mobile, hoy seria complejidad sin beneficio medible. Revisar si en el futuro se generan tipos automaticamente desde el esquema OpenAPI.
- **infra/ separado de apps/:** los Dockerfiles no viven dentro de cada app para que apps/ contenga unicamente codigo de aplicacion e infra/ concentre las decisiones de despliegue (hoy Docker Compose de desarrollo; manana, potencialmente, produccion).
- **Frontend por features, espejando los dominios del backend:** los nombres de feature (auth, realtime, historicals, reports, help, about) coinciden con los nombres de dominio backend para que ubicar "el codigo que habla de X" sea trivial en ambos lados del stack.
- **docs/decisions con ADRs numerados:** cada decision tecnica no trivial (por ejemplo, diferir la eleccion de libreria de graficos) queda registrada con su contexto y consecuencias, no solo el resultado final.

5. Codigo inicial minimo, stubs y placeholders

Con esqueleto funcional (no solo comentarios)

app/main.py, app/config/settings.py, app/websocket/manager.py, app/core/security.py (emision/validacion de JWT), app/db/session.py, el router agregador api/v1/router.py, y los modelos ORM de cada dominio (models.py, con columnas reales definidas).

Stubs explicitos (NotImplementedError o TODO)

Toda la logica de negocio (service.py de cada dominio), los endpoints REST/WS (router.py), y en el frontend casi todos los componentes de pagina — cada uno indica literalmente de que archivo del prototipo de Claude Design debe portarse (por ejemplo: "TODO: portar Login.jsx").

6. Modulos preparados desde el inicio

Los diez modulos/areas solicitados tienen carpeta propia y contrato definido desde el primer commit: autenticacion y sesion (con distincion explicita entre el proveedor de credenciales externo actual y el SSO real futuro), visualizacion en tiempo real, historicos, graficos desacoplados de la libreria final, exportacion (auditada), estado del reactor por REST y WebSocket, sugerencias y reportes de error, ayuda/documentacion, internacionalizacion (es/en con namespaces) y logs/auditoria transversal.

7. Convenciones de nombres

- **Backend:** paquetes en snake_case; cada dominio usa siempre los mismos nombres de archivo por rol (router.py, service.py, schemas.py, models.py, repository.py, dependencies.py); clases en PascalCase; constantes en UPPER_SNAKE_CASE.
- **Frontend:** componentes y paginas en PascalCase.tsx; hooks en camelCase.ts con prefijo use; servicios de API como xxxApi.ts; tipos puros en types.ts o types/*.ts; tests co-ubicados junto al archivo que prueban (*.test.tsx).

- **Iconos:** prefijo I + PascalCase (IHelp, ILogout) — convencion heredada literal del prototipo de Claude Design.
- **Reconciliacion de nombres:** el prototipo usa "histories"; el codigo usa "historicals" (mas claro en ingles para el equipo de desarrollo), mientras la etiqueta visible en pantalla sigue en espanol via i18n.

8. Integracion futura con el SSO real, sin acoplar

El pedido original mezclaba dos integraciones externas de autenticacion distintas, que quedaron separadas explicitamente en el codigo:

- **Confirmado y ya modelado:** un modulo externo e independiente valida usuario y contrasena mediante su propia API. Implementado en `app/domains/auth/adapters/external_api_provider.py`, detras de una interfaz comun `CredentialsProvider` (`adapters/base.py`).
- **Reservado, no implementado:** la integracion real con el sistema de autenticacion institucional de la UNC, mencionada en la especificacion de requerimientos como dependencia. Carpeta vacia `app/domains/auth/adapters/ssol/` con un README que documenta el contrato a cumplir.

Cambiar de proveedor de credenciales en el futuro implica editar una unica linea en `app/domains/auth/dependencies.py` (funcion `get_credentials_provider`). El frontend nunca se comunica con ningun proveedor de credenciales de forma directa — solo con el endpoint `/auth/login` del propio backend — por lo que no requiere ningun cambio cuando el backend cambie de proveedor.

9. Libreria de graficos sin acoplar el frontend

`components/charts/ChartRenderer.tsx` es el unico componente que las paginas importan para dibujar un grafico. Por detras, `activeAdapter.ts` es el unico punto de conmutacion: hoy apunta al adapter placeholder, el SVG dibujado a mano que ya traia el prototipo de Claude Design, portado tal cual. Elegir una libreria real en el futuro (Recharts, ECharts, visx, u otra) implica crear su carpeta en `adapters/`, implementar el mismo contrato de props (`types.ts::ChartRendererProps`) y cambiar esa unica linea. Ninguna pagina deberia requerir cambios. Decision registrada en `docs/decisions/0002-chart-library-deferred.md`.

10. Variables de entorno, Docker, migraciones y scripts

Variables de entorno

.env.example en la raíz (variables compartidas de Docker Compose y PostgreSQL) mas un .env.example propio por aplicacion (apps/backend, apps/frontend). Un unico modelo Settings de pydantic en el backend; los distintos entornos se resuelven cargando un archivo .env distinto, no con clases de configuracion separadas por entorno.

Docker Compose

Tres servicios para desarrollo local: db (PostgreSQL), backend (FastAPI con recarga en caliente montando el codigo como volumen) y frontend (servidor de desarrollo de Vite). No se genero un compose de produccion: ese alcance no fue confirmado.

Migraciones de base de datos

Alembic ya scaffoldado (alembic.ini, migrations/env.py, carpeta versions/ vacia) pero sin la primera migracion real generada: falta cerrar el modelo de datos definitivo, en particular el listado exacto de variables del reactor.

Scripts de desarrollo

scripts/dev/*.ps1 son wrappers cortos de Docker Compose (up, down, logs) en PowerShell, coherente con el entorno Windows del usuario. scripts/db/ contiene stubs para esperar disponibilidad de la base de datos y para poblarla con datos de desarrollo.

Documentacion tecnica inicial

docs/architecture/overview.md es el documento mas importante: contiene la lista completa de que esta confirmado por el usuario y que esta asumido a falta de definicion. docs/architecture/data-model.md documenta el modelo de datos propuesto y docs/architecture/websocket-protocol.md el contrato de topicos y eventos de WebSocket.

11. Supuestos, riesgos y evoluciones futuras

Supuestos (no confirmados explicitamente)

- Vite como build tool del frontend — solo se confirmo "React + TypeScript".
- TanStack Query para el fetching de datos REST.
- Exportacion a PNG generada del lado del cliente (canvas/SVG); el backend solo audita el evento de exportacion.
- Catalogo de ~30 variables del reactor tomado del mock del prototipo de frontend — debe confirmarse con el SIR/SSO real antes de cerrar el modelo de datos.
- Mecanismo de lectura de PostgreSQL para tiempo real (polling vs. LISTEN/NOTIFY) sin definir; impacta directamente en el requisito de latencia maxima de 2 segundos.

Riesgos de esta estructura

- La tabla ancha reactor_samples (~30 columnas) obliga a generar una migracion cada vez que se agrega, quita o renombra una variable del reactor.
- Los dominios realtime y reactor_state estan separados a proposito por responsabilidad unica, pero exige disciplina para no duplicar en ambos la logica de "esta habilitado el tiempo real".

Evoluciones futuras

- Imágenes Docker multi-stage de producción y configuración de nginx como reverse proxy (ya con carpeta reservada en `infra/nginx`).
- Redis para rate limiting o publicación/suscripción de WebSocket si la concurrencia real lo justifica.
- Tests de integración o end-to-end si el alcance del proyecto lo requiere más adelante.

Repositorio generado en D:\Facultad\Tesis (166 archivos, git inicializado, sin commits realizados). Workspace de VS Code: sse-ra0.code-workspace.